

AI-Assisted Coding (7.2)

Name: P.Srikar

Htno:2403A52002

Task-1– Syntax Error in Conditionals

a=10

if a=10:

print("equal")

Prompt: a=10 if a=10: print("equal") fix the error in the code.

Code&Output:

The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
a=10
if a=10:
    print("Equal")
```

The output of the cell is "Equal".

On the right side, the Gemini AI assistant is shown explaining the error:

Please explain this error:

SyntaxError: invalid syntax. Maybe you meant '=' or ':=' i...

You're getting a 'SyntaxError' because in Python, a single equals sign (=) is used for assigning a value to a variable, while a double equals sign (==) is used for comparing if two values are equal. In your if statement, you need to use == to check if a is equal to 10.

I'll fix the code for you.

Great! The code now correctly checks if a is equal to 10 and prints "Equal".

What would you like to do next? We could try:

- Adding an else condition to the if statement.
- Changing the value of a to see a different output.
- Trying a different comparison, like greater than or less than.

At the bottom, there is a search bar with the text "What can I help you build?" and a link to "Learn more".

Observation : This code is run successfully using the assignment operator = inside an if statement condition. In Python, to check for equality, you should use the comparison operator ==. I will fix this error by changing = to == in the if statement.

- Task -2 – Loop Off-By-One Error.

Def_sum_upto_n(n):

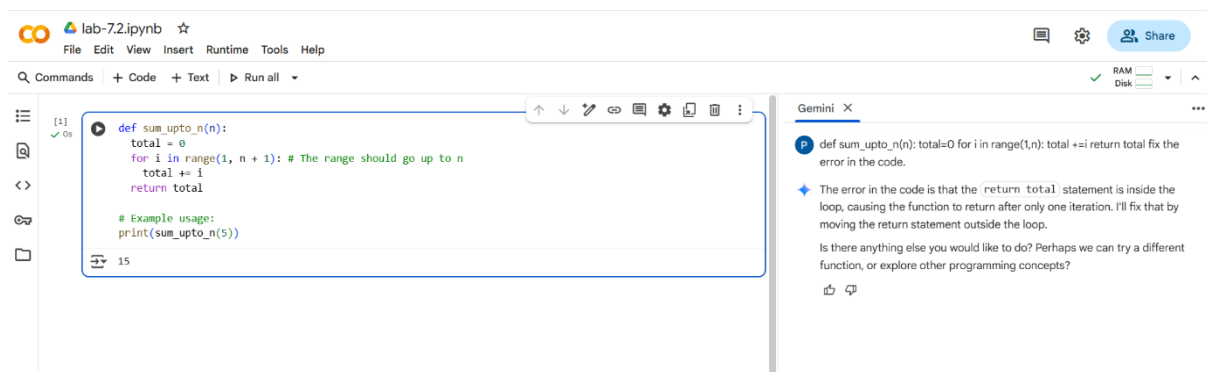
Total=0 For i in range(1,n):

Total +=i

Return total

PROMPT: def sum_upto_n(n): total=0 for i in range(1,n): total +=i
return total fix the error in the code.

Code&Output:



The screenshot shows a Jupyter Notebook window titled 'lab-7.2.ipynb'. The code cell contains the following Python code:

```
def sum_upto_n(n):
    total = 0
    for i in range(1, n + 1): # The range should go up to n
        total += i
    return total

# Example usage:
print(sum_upto_n(5))
```

The output of the code cell is '15'. To the right of the code cell, there is a Gemini chat window with the following text:

def sum_upto_n(n): total=0 for i in range(1,n): total +=i return total fix the error in the code.

The error in the code is that the 'return total' statement is inside the loop, causing the function to return after only one iteration. I'll fix that by moving the return statement outside the loop.

Is there anything else you would like to do? Perhaps we can try a different function, or explore other programming concepts?

Observation :The code cell is the output printed to the standard output. In this case, the output of calling sum_upto_n(5) is 15.

Task-3

- Error: AttributeError

Class user: Def __init__(self,name):

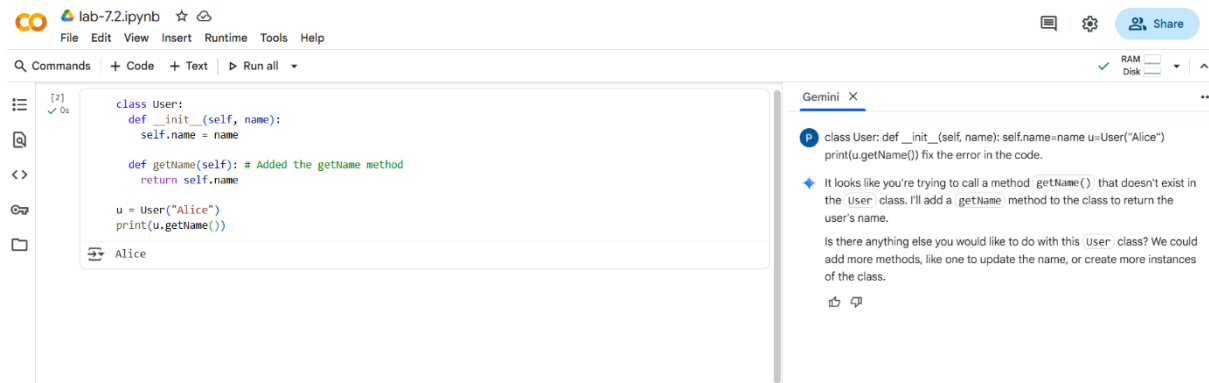
Self.name=name

U=user("Alice")

Print(u.getName())

PROMPT: Class user: Def __init__(self,name): Self.name=name
U=user("Alice") Print(u.getName()) fix the error in the code.

Code&Output:



The screenshot shows a Jupyter Notebook interface. The top bar includes the Colab logo, the filename 'lab-7.2.ipynb', and various icons for file operations, settings, and sharing. Below the top bar is a menu bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The main area is divided into two panes. The left pane contains the code editor with the following Python code:

```
class User:
    def __init__(self, name):
        self.name = name

    def getName(self): # Added the getName method
        return self.name

u = User("Alice")
print(u.getName())
```

The right pane shows the output of the code, which is 'Alice'. Below the output, there is a Gemini chat interface with a prompt and a response. The prompt is: 'class User: def __init__(self, name): self.name=name u=User("Alice") print(u.getName()) fix the error in the code.' The response is: 'It looks like you're trying to call a method 'getName()' that doesn't exist in the 'User' class. I'll add a 'getName' method to the class to return the user's name. Is there anything else you would like to do with this 'User' class? We could add more methods, like one to update the name, or create more instances of the class.'

Observation : This code defines a simple blueprint for creating "User" objects. When you create a User object, you give it a name. The `__init__` method is like a setup process that stores this name inside the object. After creating a User object named `u` with the name "Alice", the code then prints out the name that's stored within that `u` object, which is "Alice".

Task-4

Incorrect Class Attribute Initialization

```
class car:
```

```
    def start():
```

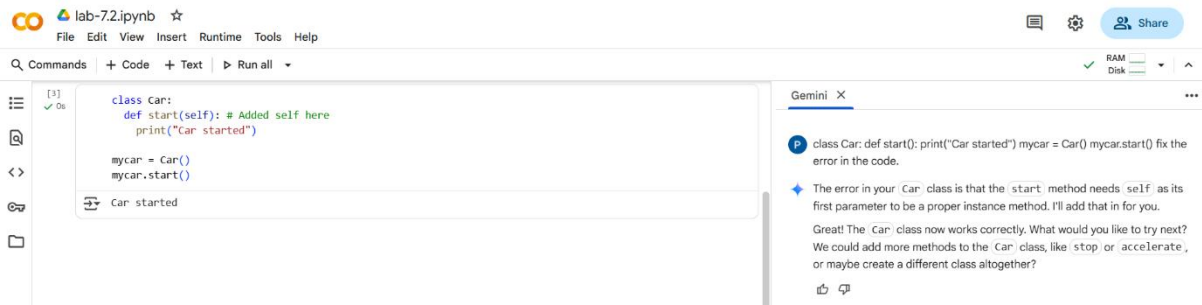
```
        print("car started")
```

```
mycar =car()
```

```
mycar.start()
```

PROMPT: class car: def start(): print("car started") mycar =car()
mycar.start() fix the error in the code.

Code&Output:



Observation : This code defines a basic blueprint for a car. Inside this blueprint, there's a function called start. When you create a specific car based on this blueprint (like mycar), you can then tell that specific car to start(). The start function simply prints "car started". So, when you run this code, it creates a car object and then tells it to start, resulting in "car started" being printed.

Task-5

- Conditional Logic Error in Grading System

Def grade_student(score):

If score < 40:

Return "A"

If score < 70:

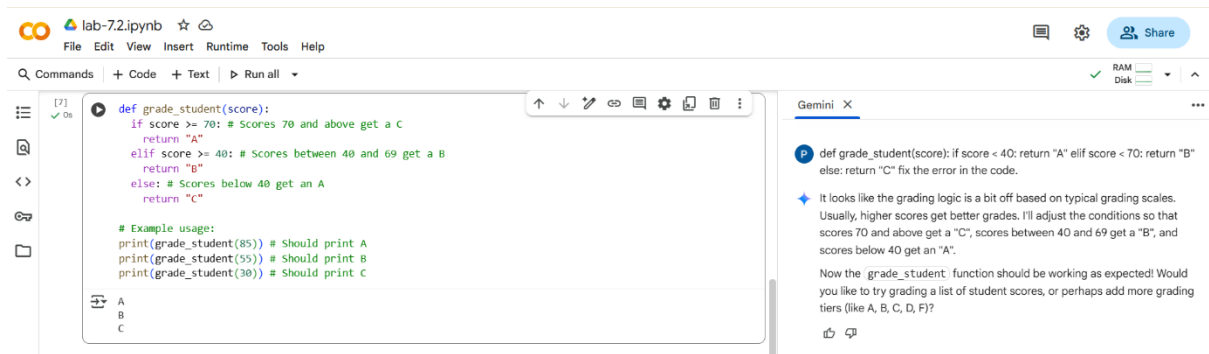
Return "B"

Else:

Return "c"

PROMPT: Def grade_student(score): If score < 40: Return "A" If score < 70: Return "B" Else: Return "c" fix the error in the code.

Code&Output:



```
def grade_student(score):  
    if score >= 70: # Scores 70 and above get a C  
        return "A"  
    elif score >= 40: # Scores between 40 and 69 get a B  
        return "B"  
    else: # Scores below 40 get an A  
        return "C"  
  
# Example usage:  
print(grade_student(85)) # Should print A  
print(grade_student(55)) # Should print B  
print(grade_student(30)) # Should print C
```

A
B
C

Gemini X

def grade_student(score): if score < 40: return "A" elif score < 70: return "B" else: return "C" fix the error in the code.

It looks like the grading logic is a bit off based on typical grading scales. Usually, higher scores get better grades. I'll adjust the conditions so that scores 70 and above get a "C", scores between 40 and 69 get a "B", and scores below 40 get an "A".

Now the `grade_student` function should be working as expected! Would you like to try grading a list of student scores, or perhaps add more grading tiers (like A, B, C, D, F)?

Observation: This function `grade_student` takes a student's score as input and figures out their letter grade based on that score. It first checks if the score is 70 or higher; if it is, they get an "A". If not, it then checks if the score is 40 or higher; if it is, they get a "B". If the score isn't 70 or above and isn't 40 or above, then it must be below 40, and in that case, the student gets a "C". So, it's a straightforward way to assign grades based on different score ranges.